SLiM Workshop Series #17: Continuous Space II

Copyright © 2020–2025 Benjamin C. Haller. All rights reserved.

1. Start with recipe 17.5 in SLiMgui

Recipe 17.5 provides a starting point that is similar to where we left off: it's a 2D continuous-space model with spatial competition (using `fitnessScaling`), nearest-neighbor mate choice (with a `mateChoice()` callback), and juvenile dispersal (in script, rather than using `deviatePositions()`, as it happens – we'll fix that below). Open recipe 17.5 from the **Open Recipe** submenu of SLiMgui's **File** menu. The model is fairly long, so it won't be reproduced here, but it should be familiar from the previous exercise.

Increase the population size from `500` to `1000`, and adjust the initial position setting with `pointUniform()` to also use `1000`.

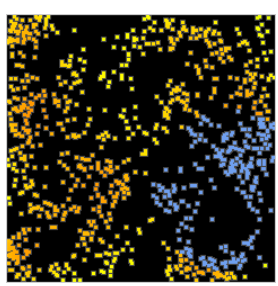
2. Add occasional natural disasters

Let's add occasional natural disasters that wipe out a part of the landscape, killing all of the individuals within a given radius of the event epicenter. This is quite easy to do. Let's start by testing the code we intend to use, by coloring the individuals of interest; this is a very useful visualization technique. Add these lines to the end of the `1: late()` event:

```
// occasional natural disasters
if (runif(1) < 0.1) {
   epicenter = p1.pointUniform();
   d = i1.distanceFromPoint(epicenter, inds);
   affected = inds[d < 0.3];
   affected.color = "cornflowerblue";
}
```

In 10% of ticks, this code chooses a disaster epicenter at random with `pointUniform()`. It then uses `i1`, which conveniently has just been evaluated for competition, to measure the distances from the epicenter to every individual (this could be done with script instead easily enough, using the Pythagorean theorem, but we might as well let `InteractionType` do it for us since it knows how). Then we select the individuals within a radius of `0.3` of the epicenter, and color them blue using the `color` property of `Individual`.

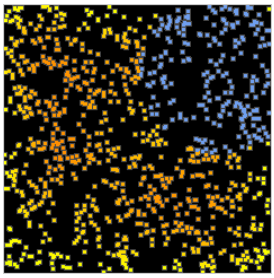
If we step forward until we get a disaster, the model looks like this:

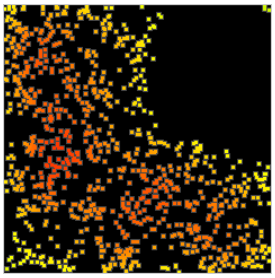


Looks good! Let's kill the individuals off afterwards, by setting their `fitnessScaling` to `0.0` in the next line:

```
affected.fitnessScaling = 0.0;
```

Now we see individuals colored blue in one tick, and then a deathly void in the next:





This is the effect we wanted, but it has revealed a deep problem with our model: it doesn't handle population density in a realistic way. Compare the two frames above. The density within the void has gone down (to zero) in the second frame, but the density across the rest of the landscape has gone *up* markedly at the same time. This is because it is a WF model, with a set population size of `1000`; if the density goes down in one area, it *has* to go up elsewhere. We would like the population size to simply drop when a disaster occurs, and then rise gradually as the void is re-colonized.

This is one example of a general phenomenon: because WF models have a fixed population size, they really struggle to behave realistically. The same would be true if a beneficial mutation were introduced at one point, for example: you would probably like the population density to be higher within the range of the spread of that mutation, reflecting lower mortality or higher fecundity, whereas areas that the mutation has not yet spread to should remain at the same density as before. With a WF model, unfortunately, the density will actually *drop* in areas outside the range of spread; if the density goes up inside the range of spread, it *has* to go down elsewhere.

These sorts of considerations, and others related to other restrictions of the WF model, mean that realistic spatial models should probably almost always be nonWF models.

3. A nonWF spatial model

Let's adapt our model to the nonWF framework. Start by adding the nonWF declaration at the top of the `initialize()` callback, with new definitions of `K` and S:

```
initializeSLiMModelType("nonWF");
defineConstant("K", 1000);
defineConstant("S", 0.1);
```

Here `K` is not exactly a carrying capacity; rather, it is what has been called a "carrying-capacity density", which tells us what the equilibrium *density* of the model would be, measured in individuals per unit area, if individuals were distributed uniformly across the landscape (and without other factors besides density-dependence influencing fitness, as discussed before). Our disasters will cause the model to drop below `K`.

The constant `S` will be used as a spatial competition kernel width. In the declaration of interaction type `i1`, **change** the maximum distance to `S*3`, and **change** the interaction function width for `i1` from `0.1` to `S`. While you're at it, **change** `i1`'s maximum strength from `3.0` to `1.0`. The reason for these changes will become clear below. (I've put these changes in **bold** because people miss them here for some reason!)

We want the neutral mutations in the model to substitute when fixed, which they won't do by default in a nonWF model, so add this line to the `initialize()` callback:

```
m1.convertToSubstitution = T;
```

Next we need to fix reproduction. Remove the `mateChoice()` callback; those are not supported in nonWF models at all, since `reproduction()` callbacks are responsible for choosing mates and generating offspring in nonWF models. Add the following `reproduction()` callback and `2: early()` event instead, after `initialize()`:

```
reproduction() {
   mate = i2.nearestNeighbors(individual, 3);
   if (!mate.size())
      return;
   mate = sample(mate, 1);
   subpop.addCrossed(individual, mate, count=rpois(1, 0.1));
}
2: early() {
   offspring = p1.subsetIndividuals(maxAge=0);
   p1.deviatePositions(offspring, "reflecting", INF, "n", 0.02);
}
```

As the `mateChoice()` callback did, the `reproduction()` callback begins by finding the three nearest neighbors of the focal individual. If no neighbors are found within the maximum interaction distance, the individual doesn't mate, so the callback returns. Otherwise, we draw one mate from the vector, and then call `addCrossed()` to generate offspring with that mate. The `count` parameter to `addCrossed()` makes life a little easier; with it, there's no need to loop over the number of offspring we want to generate.

The `2: early()` event fetches the new offspring using `subsetIndividuals()`, and then uses `deviatePositions()` to move them all. We've switched to reflecting boundaries, since we haven't tried those yet; with reflecting boundaries, an individual that tries to disperse outside of bounds "reflects" – bounces – off the edge. This code now handles juvenile dispersal, so remove the `modifyChild()` callback so that we don't move the juveniles twice.

Next, the `1 late()` and `1: late()` events should be changed to `early()` events. We want these events to run *after* offspring generation and *before* fitness recalculation; in a WF model, that means using a `late()` event, but in a nonWF model, an `early()` event is needed. The tick cycle diagrams for both model types are available in SLiMgui under the Help menu, and also on the SLiM refsheet; look at them now to see this point. This timing is needed for the `addSubpop()` call so that fitness values get calculated for the new subpopulation before it reproduces. Similarly, the `1: late()` event should be changed into a `1: early()` event because the `fitnessScaling` values it sets should take effect immediately, in the selection/survival stage of the tick cycle. We don't want individuals to get one more chance to reproduce before they die, we want them to die immediately.

There's one more fix to be made: the code doesn't actually enforce `K` anywhere. There is competition in the model, but it's not scaled to produce the carrying-capacity density we want, since this model started out as a WF model. Replace the line:

```
inds.fitnessScaling = 1.1 - competition / size(inds);
```

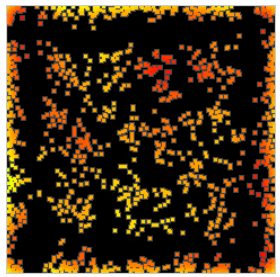
with these lines:

```
competition = (competition + 1) / (2 * PI * S^2);
inds.fitnessScaling = K / competition;
```

Instead of a relative fitness effect based on the strength of competition, we now compute an absolute fitness effect based on the local population density (as represented by the strength of competition) compared to the desired carrying-capacity density, `K`. The `+ 1` in the formula counts the focal individual itself – it is one of the individuals that count toward local population density, but it exerts no competitive interaction toward itself. The division by `(2 * PI * S^2)` basically rescales by the density of the (Gaussian) competition kernel; the SLiM manual has a long discussion of this technical point, but for our purposes here it can just be assumed. Note that the calculations are all vectorized, as usual, making them vastly more efficient. Given the rescaling here, the ratio `K / competition` is the ratio between the desired carrying-capacity density and the actual local population density; it can be used for locally density-dependent fitness, analogous to density-dependent fitness in non-spatial nonWF models.

Let's turn off natural disasters temporarily. Select the lines that impose natural disasters, and then press command-/ (forward slash). SLiMgui will comment out the selected lines.

Now recycle and run. If you made all of the above changes correctly, you should see something like this:



This seems very odd; and the population size at equilibrium seems to be around `1450` or so, as well, not `K`. What's going on?

4. Local population density

What's going on is that the individuals really like hanging out at the edge of the landscape, because they feel less competition there – the empty space outside the bounds of the landscape counts as zero density, letting the individuals pack up at higher-than-uniform density right around the edge. This high density around the edge has the knock-on effect of causing a band of low density just inside the edge area; nobody in the interior wants to be near the edge individuals because their density is so high. Further echoes of the high edge density probably continue all the way to the center, although they're harder to see.

This is also the reason for the equilibrium population size exceeding `K`: the extra individuals that can be squeezed in at the edge account for the difference. In a sense, the population density of the model at equilibrium really is `K` – if you consider the landscape as extending outward with a sort of shelf of diminishing carrying-capacity density, following the shape of the competition kernel as it extends past the edge.

Anyway, suffice to say that SLiM is doing exactly what we told it to do, but perhaps we're not happy with the result – a common outcome when developing simulations. We could fix this in a variety of ways. (a) We could decrease the target carrying-capacity density for positions near the edge of the landscape, correcting for the edge effect in the model. (b) We could make the landscape very large, so that these edge effects would (maybe) make little difference to the average behavior of the model. (c) We could increase the dispersal distance enough that these patterns would be erased by stochastic long-range dispersal (but then the model loses much of its spatiality and moves toward panmixia). Or (d) we could use periodic boundaries that wrap around at the edges to create a uniform space with no edge effects.

Fixes (a) and (d) are the most interesting; we will explore both of them. Let's pursue fix (a) first: correcting for the edge effect in the model by calculating an edge-corrected competition strength. As it happens, `InteractionType` has a method for this purpose, `localPopulationDensity()`. To use it, replace these lines:

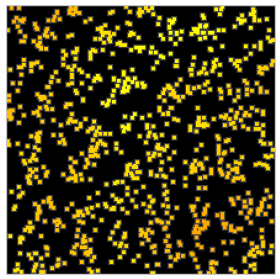
```
competition = i1.totalOfNeighborStrengths(inds);
competition = (competition + 1) / (2 * PI * S^2);
inds.fitnessScaling = K / competition;
```

with this:

```
density = i1.localPopulationDensity(inds);
inds.fitnessScaling = K / density;
```

The adjustments done by `(competition + 1) / (2 * PI * S^2)`, to change competition strengths into density values, are done internally by `localPopulationDensity()`, making the code more concise and easier to understand. More importantly, `localPopulationDensity()` corrects for the edge effect using the integral of the competition kernel *clipped to the spatial bounds*, whereas the old code used `(2 * PI * S^2)`, the unclipped integral. (Actually, `localPopulationDensity()` also clips to the maximum interaction distance for the `InteractionType`, correcting another problem with the `(2 * PI * S^2)` expression; and it works with any interaction function, not just with the Gaussian kernel. For these reasons, it is generally the preferred approach for modeling density-dependence.)

Recycle and run, and you should observe that the edge effect is indeed gone:



Much better! The equilibrium population size now varies around `1000`, as desired, too, so it looks like our problems are all solved.

5. Periodic boundaries

Now let's explore fix (d) above: using periodic boundaries, to get rid the edge effect by literally getting rid of the edges. Start by doing "undo" a couple of times, until you reach the script as it was just before the last set of changes – getting rid of the use of `localPopulationDensity()`. If reverting using undo is difficult for some reason, you can open solution `018_step03_.slim` to get back to the model at the end of section 3 above.

To implement periodic boundaries, the change needed is trivial. First, declare the periodicity by modifying the `initializeSLiMOptions()` call:

```
initializeSLiMOptions(dimensionality="xy", periodicity="xy");
```

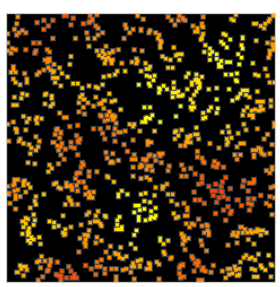
SLiM will now consider both the *x* and *y* spatial dimensions to be periodic, wrapping around at the edges to form a landscape that is, topologically speaking, a torus.

Next, change `"reflecting"`, in the call to `deviatePositions()`, to `"periodic"` instead:

```
p1.deviatePositions(offspring, "periodic", INF, "n", 0.02);
```

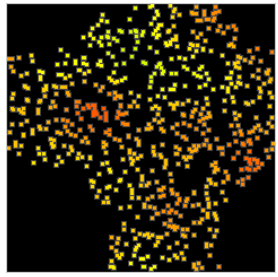
This takes a spatial point outside of bounds and brings it into bounds by wrapping it around the edges of the periodic space.

That is all that needs to be done; all spatial positions and interactions will now occur on a landscape with periodic coordinates. Recycle and run, and you should see something like this:



Again, much better – this approach has also solved our edge effect problem, and the model again equilibrates around the requested carrying-capacity density of `1000`.

Select the "natural disaster" lines and press command-/ again to uncomment them. Recycle, and then step carefully so you can observe individual natural disasters as they occur. You should notice that they now wrap around the edges of the periodic space. For example, this is a single disaster:



The blue coloring is no longer visible; since the viability/selection tick cycle stage follows directly on the heels of fitness recalculation, in nonWF models, SLiMgui never has an opportunity to show us blue individuals before they are dead and gone. Now click Play. The model might hang on by its fingernails for a little while, but it will soon go extinct; the natural disasters are frequent enough that they overwhelm the (rather low) fecundity of the model.

To compensate, change the probability of a natural disaster from `0.1` to `0.01`, then recycle and run again. Now you should see some very nice spatial dynamics. When disasters occur, the void will slowly be recolonized from the edges as new offspring are generated by the individuals on the edge. The individuals on the edges of voids generally have higher fitness, colored yellow or even green as a result of the low density in their vicinity. The population size drops naturally when a disaster occurs, and then rises as recolonization proceeds, too, as desired.

6. Habitability governed by a spatial map

Let's extend the model in one more way. Rather than the landscape being homogenous, we'd like it to vary in its habitability: some areas should support high local density, others low or zero local density. This really means varying `K` over space, instead of using one universal value of `K`.

To achieve this, add the following lines after `p1` is added and initial positions are set:

```
mapValues = matrix(sqrt(runif(100, 0, 1)), nrow=10);
map = p1.defineSpatialMap("h", "xy", mapValues, interpolate=T,
   valueRange=c(0.0, 1.0), colors=c("black", "white"));
defineConstant("MAP", map);
```

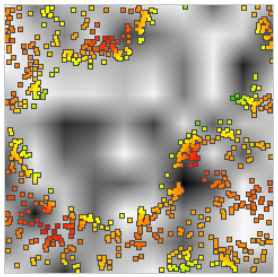
We draw `100` random values from a uniform distribution, and take their square root to bias them upward towards `1.0` (making the landscape somewhat more habitable). We convert that vector of `100` values into a 10×10 matrix with `matrix()`. This gives us the values we want to use for our spatial map, algorithmically generated. Incidentally, it is perhaps more common to load spatial map values from a file; Eidos makes this easy since it supports loading a PNG image file as an object of class `Image`, which can then be converted into a spatial map. We won't explore that here, but section 17.10 of the manual shows the approach.

Given our generated map values, we then we call the `defineSpatialMap()` method on `p1` to define a map using these values; note that spatial maps are created based upon a particular subpopulation, but they can be shared by other subpopulations also (if you are modeling multiple subpopulations occupying the same landscape, for example). We name the map `"h"` (this is mostly so it has a visible name in SLiMgui), and tell SLiM that its grid of values goes across both the *x* and *y* dimensions. We pass `mapValues` in for the values. We request (bilinear) interpolation of the map values between the

defined grid points. The last two parameters are solely for SLiMgui's benefit: we tell SLiMgui that for purposes of display, map values should be considered to range from `0.0` to `1.0` (even though the actual range of values might be narrower), and that `0.0` should be displayed as black and `1.0` as white. The colors for intermediate values will be interpolated, but you can supply more colors and color points if you wish to produce a more complex color scheme.

The `defineSpatialMap()` method returns the created map, which is an object of class `SpatialMap`. We put that into a local variable, `map`, which we then use to define a global constant, `MAP`. From this point forward, we can refer to the spatial map using the name `MAP`. This is the simplest way to work with spatial maps, although it is also possible to reference them by name (`"h"` here).

If you recycle and run, you can see the generated map behind the individuals (especially after a natural disaster clears out an area):



One problem at this point is that the map values themselves are not periodic – they ought to match at the left and right edges, and at the top and bottom edges, following the periodic boundaries of the landscape, but they don't. That's easy to fix, by making the map values at the right edge be a copy of the values at the left edge, and the values at the bottom edge be a copy of the values at the top edge; just add these lines after the definition of `mapValues`:

```
mapValues = cbind(mapValues, mapValues[,0]);
mapValues = rbind(mapValues, mapValues[0,]);
```

The map's grid is now 11×11, since we added a column and row to it. It would also be nice if the map values exhibited spatial autocorrelation, rather than each value being independent. That would produce a more natural pattern of variation across the landscape; real-world landscapes are spatially autocorrelated. There is a script in the SLiM-Extras repository for generating autocorrelated maps, which makes that easy, but we won't get into that now.

Instead, next, we want the map to actually influence habitability. This is very easy to do; replace the line:

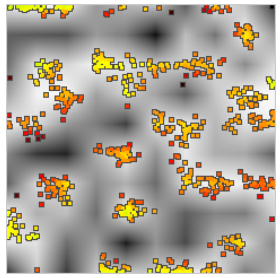
```
inds.fitnessScaling = K / competition;
```

with the lines:

```
K_local = K * MAP.mapValue(inds.spatialPosition);
inds.fitnessScaling = K_local / competition;
```

This calls the `mapValue()` method of `SpatialMap` to look up the map values at every individual's location; note that this call is vectorized, interpreting `inds.spatialPosition` correctly as a vector of points and returning the map value for each point. These map values range from `0.0` to `1.0`; we multiply by `K` to get a local carrying-capacity density, `K_local`, for each individual. Then we just use the `K_local` value for each individual, instead of `K`, to calculate the fitness effect of local density for each individual.

If we recycle and run this version of the model, we see clusters inhabiting the more clement parts of the landscape:



They mostly stay in the most habitable locations, but the connectivity of the landscape is usually sufficiently high that they can spread along high-habitability ridgelines to recolonize vacant areas.

There's one more problem left: the map's interpolation quality. When you request interpolated values from `defineSpatialMap()` with `interpolate=T`, it uses a very fast interpolation algorithm called "bilinear interpolation". This works well if map values are reasonably smooth – if they are spatially autocorrelated, as most realistic map data probably would be – but it works poorly for our random map values with no autocorrelation. In this case, bilinear interpolation between adjacent grid points with extremely different values produces "diamonds", pointy-looking spikes in the interpolated landscape. This can be seen best in the screenshot below; we'll get there.

There are better interpolation methods, and `SpatialMap` supports one: bicubic interpolation. It is much slower, so it isn't done on the fly; instead, we interpolate the map once, when we create it, to step it up to a higher resolution. Add the following lines after `defineConstant("MAP", map)`:

```
MAP.interpolate(5, "cubic");
MAP.rescale();
```

The `interpolate()` call upscales the map by a factor of `5` using bicubic interpolation, producing a 46×46 map that is more continuous. One property of bicubic interpolation is that it sometimes produces interpolated values that are outside the range of the original data, and we want our map data to be in the range [0, 1], so we call `rescale()` to scale the map's values to fit exactly into [0, 1]. Here's an example map before (left) and after (right) bicubic interpolation:

Note that we might want to leave the bilinear interpolation on; it's pretty fast, and it will interpolate the map from 46×46 up to infinite resolution for us. For maximum speed, you might want to turn it off and accept a finite-resolution map; in that case, you might use a larger rescaling factor than `5`.

There are lots of other methods to manipulate map values; see the `SpatialMap` documentation.

EXERCISE: Play around with this model and make sure you understand it fully. It's one of the most complex and sophisticated SLiM models we're going to build in this workshop, so enjoy it!

6. BEEP! With extra time, look at SLiM manual section 17.10, a very different use of spatial maps involving loading in empirical raster map data. The matrix of map values in that recipe is provided by the Eidos class `Image`, which reads in a PNG image from disk – an easy way to import map data.